Virsion control

[اكتب العنوان الفرعي للمستند]

[اكتب تلخيص المستند هنا. تلخيص المستند عبارة عن تلخيص مختصر لمحتويات المستند. اكتب تلخيص المستند هنا. تلخيص المستند عبارة عن تلخيص مختصر لمحتويات المستند.]

yasmen

١٤٤٧/٠٧/٢٦

ما هو التحكم في الإصدار؟

هو نظام يُستخدم لتتبع جميع التغييرات التي تطرأ على الملفات، خاصة ملفات التعليمات البرمجية.

يُعرف أيضًا باسم "التحكم في المصدر" أو "إدارة الكود المصدري".

يُمكّن المطورين من الرجوع إلى نسخ سابقة من الملفات في حال حدوث أخطاء.

فوائد التحكم في الإصدار للمطورين:

سجل المراجعة: يحتفظ بتاريخ كل تغيير، مما يسهل تتبع الأخطاء.

الهوية: يسجل من قام بكل تغيير، مما يعزز الشفافية والمسؤولية.

التعاون: يسمح لعدة مطورين بالعمل على نفس المشروع دون تعارض.

الأتمتة والكفاءة: يسرّع عمليات التطوير ويقلل من الأخطاء.

العمل الجماعي ومراجعة الأقران:

يمكن للمطورين إرسال تغييراتهم ليتم مراجعتها من قبل زملائهم (Peer Review).

هذه المراجعات تحسّن جودة الكود وتقلل من الأخطاء قبل دمج التغييرات في المشروع الرئيسي.

التحكم في الإصدار وDevOps

يُعد التحكم في الإصدار حجر الأساس في ممارسات DevOps.

DevOps تهدف إلى تسريع وتسليم البرمجيات بجودة عالية من خلال أدوات وممارسات متكاملة.

إذا كنت تفكر في البدء باستخدام نظام تحكم في الإصدار، فإن Git هو الخيار الأكثر شيوعًا، ويُستخدم مع منصات مثل GitHub وGitLab وBitbucket.

ملخص: التحكم في الإصدار ومنهجية العمل المرنة

التحكم في الإصدار هو أداة أساسية تُستخدم لتتبع التغييرات في الشيفرة المصدرية، وتُسهم في تحسين جودة البرمجيات وتسهيل عمليات النشر.

يعمل مطورو البرمجيات ضمن فرق متعددة التخصصات، مما يتطلب تنسيقًا عاليًا وفعالية لضمان نجاح المشروع.

غالبًا ما تُستخدم المنهجية المرنة (Agile)، حيث يُخطط الفريق ويعمل ضمن دورات زمنية قصيرة تُعرف بـ التكرارات (Iterations)، تستمر عادة لمدة أسبوعين.

تحتوي كل تكرار على قائمة مهام يجب إنجازها، بعضها قد يكون معقدًا، لكن التحكم في الإصدار يساعد في إدارتها بكفاءة.

الاختبار والأتمتة في تنفيذ المهام يعززان من كفاءة الفريق ويمنحان ثقة أكبر بأن الميزات الجديدة لن تؤثر سلبًا على سير العمل الحالي.

بعد فهم أهداف وفوائد التحكم في الإصدار، يصبح المطور مستعدًا لتعلم كيفية استخدامه بفعالية ضمن بيئة التطوير.

ثقافة التعاون في ميتا: المهندسون في موقع القيادة

في ميتا، المهندسون يقودون المشروعات، وليس مديرو المنتجات كما هو الحال في العديد من الشركات الأخرى.

المهندسون مسؤولون عن التنسيق مع علماء البيانات والباحثين، وتحديد ما يتم تصميمه ومتى.

هذا يعزز من التمكين الذاتي ويمنح المهندسين دورًا محوريًا في اتخاذ القرار.

التعاون اليومي بين الفرق

التعاون لا يقتصر على المهندسين فقط، بل يشمل علماء المنتجات والبيانات.

يتم التواصل عبر الدردشة، الاجتماعات، والمستندات التعاونية التي تُستخدم للتعليق وتبادل الملاحظات.

مثال حي: التعاون بين فرق الاتصال والمراسلة في إنستغرام نظرًا لتكامل وظائفهم.

التحكم في الإصدار في ميتا: مستودع متجانس عملاق

تستخدم ميتا مستودعًا موحدًا (Monorepo) يضم كل الكود الخاص بإنستغرام.

هذا يتيح إعادة استخدام الكود بسهولة، لكنه يخلق تحديات مثل تعارضات الدمج المتكررة.

لتقليل المخاطر:

يتم تنفيذ تغييرات صغيرة يمكن التراجع عنها بسهولة.

تُستخدم أدوات مثل "حراس البوابة" (gatekeepers) لإيقاف الميزات عند الحاجة.

تُكتب اختبارات شاملة لتجنب تعطل الخدمات مثل ماسنجر.

الشفافية والمساءلة عبر أدوات مثل blam

أداة blam تُستخدم لتحديد من كتب سطرًا معينًا من الكود.

تُسهل التواصل بين المهندسين، خاصة في بيئة ضخمة مثل ميتا حيث قد لا يعرف الجميع بعضهم البعض.

تُكتب رسائل توضيحية مع كل تغيير لتفسير الهدف منه، مما يُسهل الفهم والتراجع عند الحاجة.

دروس مستفادة من تجربة ليلى ريزفي

التعاون الفعّال هو مفتاح النجاح في بيئة العمل التقنية.

التحكم في الإصدار ليس مجرد أداة تقنية، بل هو جزء أساسي من ثقافة العمل الجماعي.

المرونة والانفتاح على تحسين كود الآخرين يعزز من جودة المنتج النهائي.

تنوع وجهات النظر حول الميزات ومتطلبات المستخدمين يؤدي إلى نتائج أفضل وأكثر شمولاً.

□ أنظمة التحكم في الإصدار والأدوات

□ ما هو التحكم في الإصدار؟

نظام يُستخدم لتتبع التعديلات على الملفات البرمجية بمرور الوقت.

يُمكّن المطورين من الرجوع إلى نسخ سابقة، مما يسهل التعاون ويقلل من تعارضات التعديلات.

□ أنواع أنظمة التحكم في الإصدار

١. CVCS – أنظمة التحكم في الإصدار المركزية

تعتمد على خادم مركزي يحتوي على المستودع الرئيسي.

المطورون يعملون كـ عملاء يسحبون ويدفعون التغييرات إلى الخادم.

الخصائص:

تحتاج إلى اتصال دائم بالخادم.

سجل التغييرات محفوظ فقط على الخادم.

أمثلة: Subversion (SVN)، Perforce، AWS CodeCommit.

٢. DVCS – أنظمة التحكم في الإصدار الموزعة

كل مطور يمتلك نسخة كاملة من المستودع على جهازه.

يمكن العمل دون اتصال بالإنترنت.

الخصائص

أداء أسرع.

مرونة أكبر في العمل الفردي.

أمثلة: Git، Mercurial.

□□ مقارنة بين CVCS وDVCS

المعيار

CVCS

DVCS

الاتصال بالخادم

ضروري لكل عملية

مطلوب فقط للسحب والدفع

سجل التغييرات

محفوظ على الخادم فقط

محفوظ محليًا وكامل

الأداء

أبطأ بسبب الاعتماد على الخادم

أسرع وأكثر مرونة

سهولة التعلم

أسهل للمبتدئين

يتطلب فهمًا أعمق

التحكم في الوصول

أكثر مركزية وتحكمًا

أكثر لامركزية

- أدوات التحكم في الإصدار الشائعة

Git: الأكثر استخدامًا حاليًا، خاصة في المشاريع مفتوحة المصدر.

Subversion (SVN): لا يزال مستخدمًا في بعض المؤسسات.

Perforce: شائع في شركات الألعاب والمشاريع الكبيرة.

Mercurial: بديل موزع لـ Git، سهل الاستخدام.

AWS CodeCommit: خدمة مستضافة من أمازون لإدارة الكود.

- الزمن: من الثمانينيات حتى اليوم

CVS (Concurrent Versions System) – 1986

أول نظام تحكم في الإصدار واسع الاستخدام.

طُوّر بواسطة والتر إف. تيشي في جامعة بوردو.

نُشر علنًا في عام ١٩٩٠.

استخدم على نطاق واسع، لكنه عانى من مشاكل مثل ضعف دعم الملفات الثنائية وعدم وجود فحوصات نزاهة.

SVN (Subversion) – 2000

طُوّر بواسطة CollabNet كبديل لـ CVS.

عالج العديد من عيوب CVS، مثل:

دعم أفضل للملفات الثنائية.

فحوصات نزاهة لضمان سلامة البيانات.

أصبح شائعًا بفضل دعم Google وSourceForge لمشاريع المصدر المفتوح.

2000 – BitKeeper

أول نظام تحكم في الإصدار موزع (Distributed VCS).

استخدمته نواة Linux لفترة.

كان مرخصًا مجانًا لـ Linus Torvalds، لكن تم سحب الترخيص في ٢٠٠٥، مما أدى إلى ظهور بدائل مفتوحة المصدر.

2005 – Mercurial وGit

Mercurial:

طُوّر بواسطة Olivia Mackall.

سهل الانتقال إليه من SVN.

تبنته منصات استضافة متعددة.

Git:

طُوّر بواسطة Linus Torvalds بعد أزمة BitKeeper.

أول إصدار عام في ٢٠٠٧.

أصبح النظام الأكثر شيوعًا بفضل:

تصميمه الموزع.

سرعته.

قوته في التعامل مع المشاريع الكبيرة مثل نواة Linux.

من النماذج المركزية إلى الموزعة

CVS وSVN يعتمدان على نموذج مركزي (Centralized VCS).

BitKeeper، Mercurial، وGit يمثلون الجيل الجديد من الأنظمة الموزعة، حيث يمتلك كل مطور نسخة كاملة من المستودع.

لماذا Git هو المهيمن اليوم؟

مرونة عالية في الفروع (branches).

أداء ممتاز مع المشاريع الضخمة.

مجتمع ضخم ودعم واسع من منصات مثل GitHub وGitLab.

من الكود إلى الإنتاج: كيف تُطلق البرامج إلى العالم

١. ✍ كتابة الكود

يبدأ المطورون بكتابة التعليمات البرمجية باستخدام محررات مثل VS Code أو IntelliJ.

تُستخدم أنظمة التحكم في الإصدار مثل Git لتتبع التغييرات والتعاون مع الفريق.

٢. مراجعة الكود (Code Review)

يتم إرسال التغييرات عبر Pull Requests.

يراجع الزملاء الكود للتأكد من جودته، أمانه، واتباعه للمعايير.

٣. الاختبار (Testing)

تُجرى اختبارات تلقائية (ويدوية أحيانًا) للتأكد من أن الكود يعمل كما هو متوقع.

تشمل اختبارات الوحدة (Unit Tests)، التكامل (Integration Tests)، والواجهة (UI Tests).

٤. □ الدمج (Integration)

بعد اجتياز المراجعة والاختبارات، يُدمج الكود في الفرع الرئيسي (main/master).

تُستخدم أدوات مثل GitHub Actions، GitLab CI/CD، أو Jenkins لأتمتة هذه الخطوة.

٥. ⍰ البناء (Build)

تُحوّل التعليمات البرمجية إلى تطبيق قابل للتنفيذ (Executable).

تُنشأ الحزم (Packages) أو الحاويات (Containers) باستخدام أدوات مثل Docker.

٦. ⍰ النشر (Deployment)

يُنشر التطبيق إلى بيئة الإنتاج (Production) باستخدام أدوات مثل Kubernetes، AWS، أو Azure.

يمكن أن يكون النشر تدريجيًا (Canary Releases) أو دفعة واحدة.

٧. ⍰ المراقبة والدعم (Monitoring & Operations)

تُراقب الخدمة باستخدام أدوات مثل Prometheus، Datadog، أو New Relic.

تُجمع تقارير الأعطال والأداء لتحسين الخدمة باستمرار.

⍰ مسرد مصطلحات Git وGitHub

⍰ أدوات وتقنيات أساسية

Git: نظام تحكم في الإصدار مفتوح المصدر وموزع، يُستخدم لتتبع التغييرات في الكود.

GitHub: منصة لاستضافة مستودعات Git والتعاون عليها، تدعم ميزات مثل المراجعة، التوثيق، وإدارة الفرق.

⍰ مفاهيم Git الأساسية

commit: لقطة (snapshot) من المستودع في لحظة معينة، تُخزن باستخدام معرف فريد (SHA).

branch: مؤشر متحرك يشير إلى commit معين، يُستخدم لتطوير ميزات أو إصلاحات بشكل منفصل.

clone: نسخة محلية كاملة من مستودع Git، تحتوي على جميع الفروع والتاريخ.

remote: مستودع مركزي (مثل الموجود على GitHub) يتشارك فيه الفريق التغييرات.

fork: نسخة من مستودع GitHub تُخزن في حساب مستخدم آخر، تُستخدم غالبًا للمساهمة في مشاريع مفتوحة المصدر.

pull request (PR): طلب دمج تغييرات من فرع إلى آخر، يُستخدم لمراجعة الكود والتعليق عليه وتشغيل الاختبارات.

HEAD: المؤشر الحالي الذي يشير إلى الفرع أو commit الذي تعمل عليه حاليًا. يمكن تغييره باستخدام git checkout.

اوامر unix:

اوامر التنقل بين الادله:

عرض الدليل الحالي: pwd يعرض المسار الكامل للدليل الذي تعمل فيه حاليًا.

عرض محتويات الدليل: ls:

يعرض أسماء الملفات والمجلدات داخل الدليل.

عرض محتويات الدليل بتنسيق قائمة مفصلة: ls –l:

يعرض معلومات إضافية مثل نوع الملف، الأذونات، المالك، المجموعة، الحجم، وتاريخ التعديل.

فهم الرموز في إخراج ls –l

d: يدل على أن العنصر هو دليل (Directory).

-: يدل على أن العنصر هو ملف عادي (Regular file).

l: يدل على أن العنصر هو رابط رمزي (Symbolic link).

التنقل بين الأدلة

الدخول إلى دليل معين:

cd etc

الرجوع إلى الدليل السابق (الأب):

cd ..

الرجوع إلى الدليل الجذر مباشرة:

cd /

الانتقال إلى مسار معين مباشرة:

cd /etc/ssh

مراجعه لما سبق من اوامر unix:

١. التحقق من الدليل الحالي:

pwd

➤ أظهر أنك في الدليل الجذري /.

٢. عرض محتويات الدليل الجذري بتنسيق مفصل:

ls -l

➤ رأيت دليلًا باسم projects.

٣. إنشاء دليل جديد باسم submissions:

mkdir submissions

٤. التحقق من إنشائه:

ls -l

٥. الدخول إلى الدليل الجديد:

cd submissions

٦. إنشاء ملفين نصيين:

touch test1.txt

touch test2.txt

٧. التحقق من وجود الملفات:

ls -l

٨. العودة إلى الدليل الجذري:

.. cd

٩. إنشاء دليل جديد باسم archive:

mkdir archive

١٠. نقل مجلد submissions إلى داخل archive:

bash mv submissions archive

١١. التحقق من نجاح النقل:

bash cd archive ls -l cd submissions ls -l

ملاحظات مفيدة:

عند استخدام cd ..، تأكد من وجود مسافة بين cd و ...

يمكنك دائمًا استخدام المسار الكامل لتجنب التنقل خطوة بخطوة، مثل:

cd /archive/submissions

لمسح الشاشة:

clear

لرؤية بنية المجلدات والملفات بشكل شجري (إن كان الأمر متاحًا):

Tree

تكمله اوامر :unix:

هذا الأمر غير صحيح نحويًا. الصيغة الصحيحة لاستخدام الأمر wc لحساب عدد الكلمات في ملف هي:

wc -w file1.txt

أو باستخدام ممر البيانات (pipe):

cat file1.txt | wc -w

وبنفس الطريقة، لحساب عدد الكلمات في ملفين معًا:

```
cat file1.txt file2.txt | wc -w
```

شرح سريع للأوامر:

ls: يعرض محتويات المجلد الحالي.

cd archive: يدخل إلى مجلد archive.

cat file1.txt: يعرض محتوى الملف.

wc -w file1.txt: يحسب عدد الكلمات في الملف.

| (ممر البيانات): يمرر ناتج أمر إلى أمر آخر.

ساسيات إعادة التوجيه في Linux

١. أنواع التدفقات (Streams)

٠ - stdin (المدخلات القياسية): عادةً ما تأتي من لوحة المفاتيح.

١ - stdout (المخرجات القياسية): تُعرض على الشاشة.

٢ - stderr (الخطأ القياسي): تُعرض أيضًا على الشاشة، لكنها مخصصة للأخطاء.

إعادة توجيه المدخلات (Input Redirection)

الرمز المستخدم: <

مثال:

```
cat < input.txt
```

هذا يقرأ محتوى input.txt ويعرضه.

لإنشاء ملف من مدخلات المستخدم:

```
cat > input.txt
```

ثم تكتب النص، وتنهي بـ Ctrl + D.

إعادة توجيه المخرجات (Output Redirection)

الرمز المستخدم: >

مثال:

```
ls -l > output.txt
```

هذا يحفظ ناتج الأمر ls -l في الملف output.txt.

لإلحاق المخرجات دون الكتابة فوق الملف:

```
ls -l >> output.txt
```

□□ إعادة توجيه الأخطاء (Error Redirection)

الرمز المستخدم: ٢>

مثال:

```
ls /bin/us > output.txt 2> error.txt
```

هذا يحفظ المخرجات في output.txt، والأخطاء في error.txt.

لدمج المخرجات والأخطاء في ملف واحد:

```
ls /bin/us > all.txt 2>&1
```

ملاحظات مهمة

كل شيء في Linux يُعامل كملف، بما في ذلك الأجهزة والمخرجات.

إعادة التوجيه تمنحك تحكمًا كبيرًا في سير العمل، خاصة عند كتابة سكربتات أو التعامل مع ملفات كبيرة.

⍰ ما هي أداة grep؟

الاسم: اختصار لـ Global Regular Expression Print.

الوظيفة: البحث عن أنماط نصية داخل الملفات أو المخرجات.

الأساس: تعتمد على التعبيرات العادية (Regex) لتحديد الأنماط.

⍰ أمثلة عملية على استخدام grep

١. بحث بسيط عن كلمة

```
grep Sam names.txt
```

يبحث عن كل الأسطر التي تحتوي على الكلمة "Sam".

حساس لحالة الأحرف (لن يعثر على "sam" أو "sAm").

٢. تجاهل حساسية حالة الأحرف

grep -i Sam names.txt

يعرض كل الأسطر التي تحتوي على "Sam" أو "sam" أو أي شكل آخر من الكلمة بغض النظر عن حالة الأحرف.

٣. مطابقة الكلمة بالكامل فقط

grep -w Sam names.txt

يعرض فقط الأسطر التي تحتوي على الكلمة "Sam" ككلمة مستقلة، ويتجاهل الكلمات مثل "Samuel" أو "Samantha".

٤. استخدام ممر البيانات (Pipe) مع grep

ls /bin | grep zip

يعرض فقط الملفات داخل /bin التي تحتوي على الكلمة "zip" في اسمها.

٥. دمج خيارات متعددة

ls /bin | grep -iw zip

يبحث عن "zip" مع تجاهل حالة الأحرف (-i) ومطابقة الكلمة بالكامل (-w).

نصائح إضافية لاستخدام grep

لإظهار رقم السطر:

grep -n Sam names.txt

يعرض رقم السطر بجانب كل نتيجة.

لعرض عدد مرات الظهور فقط:

grep -c Sam names.txt

للبحث في مجلد كامل:

grep -r Sam /path/to/directory

لإظهار الأسطر التي لا تحتوي على النمط:

grep -v Sam names.txt

ما هو Git؟

Git هو نظام تحكم في الإصدارات (Version Control System) يُستخدم لتتبع التغييرات التي تطرأ على الملفات، خاصة في مشاريع البرمجة. تم تطويره بواسطة لينوس تورفالدس في عام ٢٠٠٥ لإدارة تطوير نواة نظام لينكس، وهو الآن الأداة الأكثر استخدامًا لهذا الغرض.

ميزات Git:

تتبع التغييرات: يمكنك الرجوع لأي نسخة سابقة من ملفاتك.

العمل الجماعي: يسمح لعدة مطورين بالعمل على نفس المشروع دون تعارض.

السرعة والكفاءة: سريع جدًا في تنفيذ الأوامر.

العمل بدون اتصال: معظم العمليات يمكن تنفيذها محليًا دون الحاجة للإنترنت.

مفتوح المصدر: مجاني ويمكن تعديله حسب الحاجة

☘ ما هو GitHub؟

GitHub هو منصة استضافة سحابية لمستودعات Git. بمعنى آخر، هو المكان الذي يمكنك فيه رفع مشاريعك التي تستخدم Git ومشاركتها مع الآخرين أو العمل عليها بشكل جماعي.

 ميزات GitHub:

واجهة رسومية سهلة لإدارة مستودعات Git.

طلبات السحب (Pull Requests): لاقتراح تغييرات ومراجعتها قبل دمجها.

إدارة الصلاحيات: لتحديد من يمكنه التعديل أو المساهمة.

التوثيق والتذاكر: لتوثيق المشروع وتتبع المهام والمشكلات.

التكامل مع أدوات CI/CD: لأتمتة الاختبارات والنشر.

شبكة اجتماعية للمطورين: يمكنك متابعة الآخرين، النجمة للمشاريع، والمساهمة في مشاريع مفتوحة المصدر.

الفرق بين Git و GitHub:

Git

GitHub

أداة لإدارة الإصدارات محليًا

منصة سحابية لاستضافة مستودعات Git

يعمل عبر سطر الأوامر

يوفر واجهة رسومية سهلة الاستخدام

لا يتطلب اتصالًا دائمًا بالإنترنت

يتطلب الإنترنت للوصول إلى المستودعات

مفتوح المصدر ومجاني

يقدم خططًا مجانية ومدفوعة حسب الاستخدام

 لماذا يستخدمه المطورون؟

لتجنب فقدان العمل أو الكتابة فوق التعديلات.

لتسهيل التعاون بين أعضاء الفريق.

لتوثيق تاريخ المشروع وتطوره.

للمساهمة في مشاريع مفتوحة المصدر.

□ خطوات إنشاء مستودع على GitHub واستنساخه باستخدام Git

١. إنشاء مستودع جديد على GitHub

تسجيل الدخول إلى حسابك على GitHub.

النقر على الزر الأخضر "New repository".

اختيار المالك (عادةً حسابك الشخصي).

إدخال اسم المستودع (مثلاً: my-first-repo).

التأكد من أن الاسم متاح (ظهور علامة □).

إدخال وصف للمستودع (مثلاً: "حساب تدريب لتعلم Git").

اختيار ما إذا كان المستودع عامًا (Public) أو خاصًا (Private).

تفعيل خيار إنشاء ملف README.

النقر على "Create repository".

٢. استنساخ المستودع باستخدام HTTPS

النقر على زر "Code" ثم اختيار HTTPS.

النقر على "Copy" لنسخ رابط المستودع.

فتح سطر الأوامر (Terminal).

إنشاء مجلد جديد للمشروعات:

```
mkdir projects
cd projects
```

استنساخ المستودع:

```
git clone https://github.com/username/my-first-repo.git
```

الانتقال إلى مجلد المستودع:

```
cd my-first-repo
```

عرض الملفات:

```
ls -la
```

٣. ما الذي ستجده داخل المستودع؟

ملف README.md: ملف Markdown يُستخدم لتوثيق المشروع.

مجلد .git: يحتوي على جميع بيانات Git الخاصة بالمستودع (لا تلمسه يدويًا!).

الفرع الرئيسي main: هو الخط الزمني الأساسي للمشروع.

□ ملاحظات مهمة:

يمكنك لاحقًا إضافة ملفات جديدة، تعديل الملفات الحالية، ثم استخدام أوامر مثل:

```
git add .
git commit -m "رسالة التعديل"
git push origin main
```

▯ مراحل سير العمل في Git (Git Workflow)

١. Modified – تم التعديل

أي تغيير تقوم به على ملف موجود أو تضيف ملفًا جديدًا داخل المستودع.

Git يلاحظ التغيير، لكنه لا يتتبعه بعد.

مثال:

```
echo "مرحبا Git" > hello.txt
```

٢. Staged – تم الإعداد

تُضيف الملف إلى منطقة الإعداد (Staging Area) باستخدام الأمر:

```
git add hello.txt
```

هذا يعني أنك تقول لـ Git: "أنا جاهز لتضمين هذا التغيير في الالتزام القادم".

٣. Committed – تم الالتزام

تُسجل التغييرات في سجل Git باستخدام:

```
git commit -m "أضفت ملف hello.txt"
```

هذا يُنشئ نقطة حفظ يمكنك الرجوع إليها لاحقًا.

دفع التغييرات إلى GitHub (المستودع البعيد)

بعد الالتزام، يمكنك دفع التغييرات إلى GitHub باستخدام:

```
git push origin main
```

origin: هو اسم المستودع البعيد (GitHub).

main: هو اسم الفرع الرئيسي.

▯ سحب التغييرات من GitHub (Pull)

إذا قام شخص آخر بإجراء تغييرات على نفس المستودع، يمكنك سحب التحديثات إلى جهازك المحلي باستخدام:

```
git pull origin main
```

▯ مثال عملي كامل:

```
cd my-first-repo
echo "سطر جديد في الملف" >> README.md
git status
git add README.md
git commit -m "تحديث README.md"
git push origin main
```

▯ ماذا يوجد داخل مجلد .git؟

يحتوي على كل البيانات التي يستخدمها Git لتتبع التغييرات.

لا يجب التعديل عليه يدويًا.

يمكنك استكشافه لاحقًا لفهم كيفية عمل Git داخليًا.

مفاهيم مهمة من تجربتك:

شجرة العمل النظيفة تعني أنه لا توجد تغييرات غير ملتزم بها.

Git موزع: كل شيء يتم محليًا أولًا، ولا يُرسل إلى GitHub إلا عند تنفيذ git push.

منطقة الإعداد تمنحك تحكمًا دقيقًا في ما تريد تضمينه في كل التزام.

سير العمل الذي اتبعته

١. التحقق من المسار الحالي باستخدام pwd والتأكد من أنك في الدليل الصحيح.

٢. التحقق من حالة المستودع باستخدام git status.

٣. إنشاء فرع جديد باستخدام:

git checkout -b feature/lesson-highlight (إنشاء + انتقال للفرع)، أو

git branch feature/lesson-highlight ثم git checkout feature/lesson-highlight.

٤. التحقق من الفروع الحالية باستخدام git branch.

٥. إضافة ملف جديد باستخدام:

```
touch test2.txt
git add test2.txt
"git commit -m "Add test2.txt
```

٦. دفع الفرع الجديد إلى GitHub باستخدام:

```
git push -u origin feature/lesson-highlight
```

٧. فتح GitHub وإنشاء Pull Request لمراجعة التغييرات ودمجها مع الفرع الرئيسي.

٨. دمج الفرع بعد الموافقة، مع خيار حذف الفرع أو الاحتفاظ به.

٩. العودة إلى الفرع الرئيسي باستخدام:

```
git checkout main
```

git pull لجلب التغييرات المدمجة.

١٠. التحقق من الملفات باستخدام ls للتأكد من وجود test2.txt.

نصائح إضافية

تسمية الفروع: من الأفضل استخدام تسميات واضحة ومنظمة مثل:

feature/اسم-الميزة

bugfix/اسم-الإصلاح

hotfix/اسم-الإصلاح-العاجل

refactor/اسم-التحسين

العمل الجماعي: حتى لو لم يكن لديك مراجعون الآن، من الجيد أن تعتاد على إنشاء Pull Requests، لأنها توثق التغييرات وتُسهل المراجعة لاحقًا.

الدمج الآمن: قبل الدمج، تأكد دائمًا من أن الفرع الرئيسي محدث (git pull origin main) لتجنب التعارضات.

١. التحقق من المسار الحالي

الأمر: pwd

النتيجة: تأكدت أنك في الدليل my-first-repo.

٢. التحقق من حالة المستودع

الأمر: git status

الهدف: التأكد من عدم وجود تغييرات غير ملتزمة (unstaged changes) وأن الفرع محدث مع الأصل (origin/main).

٣. إنشاء فرع جديد

الطريقة ١: git checkout -b lesson-feature

ينشئ الفرع وينقلك إليه مباشرة.

الطريقة ٢:

git branch lesson-feature (إنشاء فقط)

git checkout lesson-feature (الانتقال إليه)

٤. التحقق من الفرع الحالي

الأمر: git branch

النتيجة: يظهر لك جميع الفروع، مع تمييز الفرع النشط بعلامة *.

٥. إضافة ملف جديد

الأمر: touch test2.txt

٦. إضافة الملف إلى مرحلة التتبع

الأمر: git add test2.txt

٧. تسجيل الالتزام (commit)

الأمر: "git commit -m "Add test2.txt

٨. دفع الفرع الجديد إلى GitHub

الأمر: git push -u origin lesson-feature

-u تعني تعيين هذا الفرع كفرع التتبع الافتراضي للفرع البعيد.

٩. إنشاء طلب سحب (Pull Request)

من خلال GitHub:

افتح المستودع.
سترى إشعارًا بوجود فرع جديد.
انقر على "Compare & pull request".
راجع التغييرات، ثم انقر على "Create pull request".

١٠. مراجعة الفريق ودمج التغييرات

الفريق يراجع التغييرات.

بعد الموافقة، يتم دمج الفرع مع main.

لماذا هذه الممارسة مهمة؟

تنظيم العمل الجماعي: كل شخص يعمل على فرع مستقل.

تقليل الأخطاء: لا يتم دمج التغييرات إلا بعد مراجعتها.

سهولة التتبع: كل ميزة أو إصلاح يتم تتبعه في فرع منفصل.

مرونة التطوير: يمكن العمل على ميزات متعددة بالتوازي دون تعارض.

 الفرق بين المستودع المحلي والبعيد في Git

النوع

الوصف

محلي (Local)

المستودع الموجود على جهازك الشخصي. تعمل عليه دون الحاجة للاتصال بالإنترنت.

بعيد (Remote)

مستودع موجود على خادم خارجي (مثل GitHub). يُستخدم لمشاركة الكود مع الفريق.

 مصطلحات أساسية في Git

Clone (استنساخ): نسخ مستودع بعيد إلى جهازك المحلي.

Pull (سحب): جلب التغييرات من المستودع البعيد إلى المحلي.

Push (دفع): إرسال التغييرات من المحلي إلى البعيد.

Remote: عنوان URL يشير إلى المستودع البعيد.

Branch: فرع من الكود يمكن العمل عليه بشكل مستقل.

 خطوات إعداد مستودع محلي وربطه بمستودع بعيد

١. إنشاء مستودع محلي جديد

```
mkdir my-second-repo
cd my-second-repo
git init
```

٢. التحقق من إعدادات المستودع البعيد (لا يوجد بعد)

```
git remote -v
```

٣. إضافة مستودع بعيد (GitHub)

```
git remote add origin git@github.com:GitTutorials101/my-first-repo.git
```

٤. التحقق من الإعداد

```
git remote -v
```

٥. سحب التغييرات من المستودع البعيد

```
git pull origin main
```

إذا لم يكن لديك فرع main محليًا، أنشئه وتتبعه:

```
git checkout -b main origin/main
```

٦. التحقق من الملفات

```
ls
```

□ مميزات هذا الأسلوب

العمل دون اتصال: يمكنك إجراء تغييرات محليًا ثم دفعها لاحقًا.

التعاون السلس: يتيح للمطورين العمل على نفس المشروع دون تعارض.

التحكم الكامل: يمكنك تتبع كل تغيير، والعودة لأي نقطة سابقة.

□ ما الذي يحدث عند استخدام git pull؟

١. الاتصال بالمستودع البعيد (Remote Repository)

Git يتصل بعنوان URL المرتبط بالمستودع البعيد (عادةً يسمى origin).

٢. جلب التغييرات (Fetch):

يتم تحميل التغييرات الجديدة من الفرع البعيد (مثل origin/main) إلى جهازك المحلي، دون دمجها بعد.

٣. دمج التغييرات (Merge):

Git يحاول دمج التغييرات من الفرع البعيد مع الفرع المحلي الحالي.

إذا لم تكن هناك تعارضات، يتم الدمج تلقائيًا.

إذا وُجدت تعارضات، سيطلب منك Git حلها يدويًا.

مثال عملي

```
# التحقق من الفرع الحالي
git branch
# التأكد من الحالة
git status
# سحب التغييرات من الفرع البعيد
git pull origin main
# التحقق من التغييرات
cat test.txt
```

نصائح مهمة

قبل git push، نفّذ دائمًا git pull لتقليل فرص التعارضات.

استخدم git fetch إذا أردت فقط جلب التغييرات دون دمجها فورًا.

احرص على العمل في الفرع الصحيح قبل تنفيذ أي أوامر دفع أو سحب.

راجع التغييرات باستخدام git log أو git diff لتفهم ما الذي تغير.

المفاهيم الأساسية

git add: يضيف التغييرات إلى منطقة التقسيم المرحلي (staging area).

git commit: يحفظ التغييرات في المستودع المحلي.

git push: يرسل التغييرات من المستودع المحلي إلى المستودع البعيد (GitHub).

git pull: يجلب التغييرات من المستودع البعيد إلى المحلي ويقوم بدمجها.

□ خطوات العمل النموذجية

١. التحقق من الحالة والفرع الحالي

```
git status
git branch
```

٢. إضافة التغييرات والالتزام بها

```
. git add
git commit -m "وصف التغييرات"
```

٣. دفع التغييرات إلى المستودع البعيد

```
git push origin main
```

إذا كنت تستخدم HTTPS، سيُطلب منك إدخال بيانات تسجيل الدخول إلى GitHub.

٤. التحقق من التغييرات على GitHub

افتح المستودع على GitHub.

تحقق من ظهور الملف الجديد أو التعديلات في واجهة المستخدم.

□ التعامل مع التغييرات من المطورين الآخرين

٥. سحب التغييرات من المستودع البعيد

```
git pull origin main
```

من الأفضل دائمًا تنفيذ git pull قبل git push لتقليل احتمالات التعارضات.

٦. التحقق من الملفات بعد السحب

```
cat test.txt
```

سترى التعديلات التي أُجريت على GitHub (مثل إضافة "تغير المناخ").

□□ ملاحظات مهمة

الدمج التلقائي: إذا لم تكن هناك تعارضات، سيتم دمج التغييرات تلقائيًا.

التعارضات (Conflicts): إذا عدّل أكثر من شخص نفس السطر في الملف، سيطلب Git منك حل التعارض يدويًا.

العمل الجماعي: git pull وgit push هما أساس التعاون في Git، ويجب استخدامهما بانتظام.

⍰ مفهوم العنوان (HEAD) في Git

HEAD هو مؤشر خاص داخل مجلد .git يشير إلى آخر التزام (commit) تعمل عليه حاليًا.

يوجد هذا المؤشر في الملف .git/HEAD، ويحتوي على مرجع إلى الفرع النشط (مثل refs/heads/main).

□ كيفية التحقق من الفرع الحالي والالتزام المرتبط به

لعرض الفرع الحالي:

```
git branch
```

لعرض محتوى ملف HEAD:

```
cat .git/HEAD
```

إذا كنت على الفرع الرئيسي، سترى شيئًا مثل:

```
ref: refs/heads/main
```

⍰ تبديل الفروع وتحديث HEAD

عند تنفيذ الأمر:

```
git checkout testing
```

يتم تحديث ملف HEAD ليشير إلى:

```
ref: refs/heads/testing
```

هذا يعني أن HEAD الآن يتتبع الفرع testing بدلًا من main.

معرف الالتزام (Commit Hash)

كل التزام في Git يُمثّل بمعرف فريد (SHA-1 hash).

يمكن عرض معرف الالتزام الحالي للفرع باستخدام:

```
cat .git/refs/heads/main
```

عند إجراء تعديل على ملف (مثل README.md) ثم الالتزام به، يتغير هذا المعرف ليعكس التزامًا جديدًا.

مثال عملي على تغيير الالتزام

١. تعديل ملف README.md باستخدام محرر مثل vim.

٢. إضافة الملف:

```
git add .
```

٣. الالتزام بالتغيير:

```
git commit -m "تحديث بسيط على README"
```

٤. التحقق من تغير معرف الالتزام:

```
cat .git/refs/heads/main
```

سترى أن المعرف قد تغير، مما يدل على أن HEAD يشير الآن إلى التزام جديد.

خلاصة المفهوم

HEAD هو البوصلة التي تحدد موقعك الحالي في سجل التزامات المشروع.

عند كل التزام جديد، يتم تحديث المعرف الذي يشير إليه HEAD ليعكس أحدث حالة للمشروع.

فهم كيفية عمل HEAD ضروري للتنقل بين الفروع وتتبع التعديلات بدقة.

المجلد المخفي .git ودوره

يوجد في كل مشروع Git ويحتوي على جميع البيانات اللازمة لتتبع التغييرات.

يتضمن ملفًا مهمًا يسمى HEAD، وهو مؤشر يشير إلى الفرع الحالي والالتزام (commit) النشط.

كيفية معرفة الفرع والالتزام الحالي

باستخدام الأمر cat .git/HEAD يمكن معرفة الفرع الذي تعمل عليه حاليًا.

يشير هذا الملف إلى مسار مثل refs/heads/main، والذي يحتوي على معرف الالتزام الحالي (SHA-1 hash).

يمكن عرض هذا المعرف باستخدام الأمر cat .git/refs/heads/main.

التنقل بين الفروع وتأثيره على HEAD

عند تنفيذ git checkout branch-name، يتغير محتوى HEAD ليشير إلى الفرع الجديد.

يمكن التحقق من التغيير باستخدام cat .git/HEAD مرة أخرى.

هذا يوضح كيف يتتبع Git الفرع النشط في كل لحظة.

تجربة عملية لتغيير الفرع والتحقق من التحديثات

يتم إنشاء فرع جديد باستخدام git checkout feature/testing-branches.

بعد التبديل، يتغير HEAD ليشير إلى refs/heads/feature/testing-branches.

العودة إلى الفرع الرئيسي تتم بـ git checkout main.

✍ تعديل الملفات ومراقبة التغييرات

عند تعديل ملف مثل README.md باستخدام محرر مثل Vim أو VS Code، لا يتغير معرف الالتزام حتى يتم تنفيذ commit.

يمكن التحقق من ذلك بمقارنة معرف الالتزام قبل وبعد التعديل باستخدام cat .git/refs/heads/main.

تنفيذ التعديلات وتحديث سجل Git

١. إضافة الملف المعدل: git add . أو git add README.md

٢. الالتزام بالتغيير: git commit -m "رسالة توضيحية"

٣. التحقق من التغيير: يظهر معرف جديد في refs/heads/main، مما يدل على أن الالتزام قد تم.

الفكرة الأساسية

Git يتتبع كل تغيير بدقة من خلال نظام إشارات داخلي يعتمد على ملفات داخل .git. فهم كيفية عمل HEAD والفروع والالتزامات يمنح المطورين تحكمًا كاملاً في تاريخ المشروع وتطوره.

البرمجة تشبه كتابة الروايات: تحتاج إلى مراجعة مستمرة، وقد تكون هناك أفكار أو تغييرات سابقة تستحق العودة إليها.

Git يوفر أدوات قوية لمتابعة التعديلات، وأبرزها الأمر git diff الذي يُستخدم لمقارنة التغييرات بين الملفات أو الفروع أو الالتزامات.

أدوات Git للمقارنة

git status: يعرض الملفات التي تم تعديلها أو إضافتها أو حذفها.

git diff: يُظهر تفاصيل التغييرات داخل الملفات، مثل الأسطر التي أُضيفت أو حُذفت.

مثال توضيحي باستخدام ملف CITIES.txt

إذا قمت بتحديث الملف أثناء السفر ثم فقدت بعض البيانات، يمكنك استخدام git diff لمقارنة النسخة الحالية مع النسخة السابقة.

يُظهر الأمر الأسطر المحذوفة (مع علامة -) والمضافة (مع علامة +).

تعديل ملف ومقارنته قبل الالتزام

١. تعديل ملف مثل README.md باستخدام محرر نصوص.

٢. قبل الالتزام، يمكن استخدام:

```
git diff HEAD README.md
```

لمقارنة التغييرات مع آخر التزام.

مقارنة بين الالتزامات

استخدم git log --oneline لعرض سجل الالتزامات بشكل مختصر.

ثم استخدم:

```
git diff commit1 commit2
```

لمقارنة التغييرات بين التزامين محددين.

مقارنة بين الفروع

اعرض الفروع باستخدام:

```
git branch
```

ثم قارن بين فرعين باستخدام:

```
git diff main feature-branch
```

لرؤية الفروقات بين الفرع الرئيسي والفرع المميز.

الفكرة العامة

عند إدارة فريق كبير من المطورين، يصبح من الصعب تتبع من قام بتعديل ماذا ومتى.

git blame

Git يوفر أداة قوية تسمى git blame تساعدك على معرفة من أجرى كل تغيير في كل سطر من الملف، ومتى تم ذلك.

وظيفة git blame

يعرض معلومات تفصيلية لكل سطر في الملف، تشمل:

معرف الالتزام (commit hash): يشير إلى التزام معين.

اسم المؤلف: المطور الذي أجرى التغيير.

الطابع الزمني: تاريخ ووقت التعديل.

رقم السطر: موقع التغيير داخل الملف.

المحتوى: السطر البرمجي نفسه.

كيفية استخدام git blame

الصيغة الأساسية:

```
git blame اسم_الملف
```

مثال:

```
git blame feature.js
```

يعرض كل سطر في الملف مع تفاصيل من قام بتعديله ومتى.

تحليل مخرجات git blame

كل سطر يبدأ بـ:

معرف الالتزام (hash)

اسم المطور

تاريخ ووقت التعديل

رقم السطر

السطر البرمجي المعدل

إذا تكرر نفس المعرف في عدة أسطر، فهذا يعني أن التزامًا واحدًا غيّر عدة أسطر.

مثال عملي باستخدام مستودع MK Docs

تم استخدام ملف setup.py كمثال واقعي.

الأمر:

```
git blame setup.py
```

يعرض جميع التعديلات التي أجراها المطورون المختلفون على الملف.

✂ تقطيع المخرجات حسب الأسط

لتقليل حجم المخرجات في الملفات الكبيرة، يمكن تحديد نطاق أسطر:

```
git blame -L 5,15 setup.py
```

يعرض فقط التغييرات بين السطرين ٥ و١٥.

تخصيص المخرجات

يمكن استخدام خيارات إضافية لتغيير تنسيق العرض، مثل:

```
git blame -l setup.py
```

يعرض معرفات الالتزام بشكل أطول أو بتنسيقات مختلفة.

ما هو git blame؟

أداة قوية في Git تُستخدم لتتبع من أجرى كل تغيير في ملف معين، ومتى تم ذلك.

تعرض معلومات مفصلة عن كل سطر في الملف، بما في ذلك:

معرّف الالتزام (SHA): يمثل التزام Git الذي أُجري فيه التغيير.

اسم المؤلف: المطور الذي أجرى التعديل.

الطابع الزمني: تاريخ ووقت التغيير.

رقم السطر: موقع التغيير في الملف.

المحتوى: السطر البرمجي نفسه.

□ كيفية استخدام git blame

الصيغة الأساسية:

```
git blame filename
```

مثال: git blame feature.js

يعرض كل سطر من الملف مع تفاصيل من أجرى التغيير ومتى.

□ تحليل المخرجات

كل سطر يبدأ بـ:

SHA الكامل أو المختصر (يمكن التحكم في طوله).

اسم المؤلف.

تاريخ ووقت التعديل.

رقم السطر.

السطر البرمجي المعدل.

✂□ تحديد نطاق الأسطر

لتقليل حجم المخرجات عند التعامل مع ملفات كبيرة:

```
git blame -L 5,15 filename
```

يعرض فقط الأسطر من ٥ إلى ١٥.

□□ تخصيص المخرجا

يمكن استخدام خيارات إضافية لتغيير طريقة عرض

عرض SHA الكامل.

إظهار البريد الإلكتروني للمؤلف.

تعديل تنسيق التاريخ.

□ الربط مع git log

git blame يحدد أين حدث التغيير.

للحصول على تفاصيل التغيير، استخدم:

```
git log -p SHA
```

يعرض التعديلات التي تمت في ذلك الالتزام.

يُفضل استخدام git blame وgit log معًا لفهم التغييرات بعمق.

The End